

CUBE BUILDATHON

ROUND 3  
PARTICIPANT HANDBOOK

Pod Integration Build | Commerce Context

INTEGRATE → ORCHESTRATE → TEST → DEPLOY → DEMONSTRATE

| RECEIVING             | PREP / PACK                 | RETURNS                    | RECOVERY                   | FINAL OUTCOME      |
|-----------------------|-----------------------------|----------------------------|----------------------------|--------------------|
| Inspect inbound units | Route by fulfilment channel | Process only when returned | Compare fees with evidence | Traceable decision |

**Your goal is not to build five separate demos.**  
**Your goal is one connected, testable commerce system.**

Prepared from the Round 3 starter repository and competition guidance  
5 participants per Pod • One shared repository • One evidence trail

OFFICIAL STARTER REPOSITORY

[github.com/Cube-Build-A-Thon/cube-round3-pod](https://github.com/Cube-Build-A-Thon/cube-round3-pod)

# How to use this handbook

This handbook is the participant-friendly route through the Round 3 build. The linked repository remains the source of truth for code, schemas, detailed rules and examples. Read this first, then follow the repository guides listed in Section 14.

| If you need to...                             | Start here                             |
|-----------------------------------------------|----------------------------------------|
| Get the system running                        | Section 2: First 30 minutes            |
| Understand your Pod type                      | Section 3: Standard vs Specialist Pods |
| Integrate your agent                          | Sections 4–6                           |
| Build the orchestrator                        | Section 7                              |
| Handle uncertainty, failures and missing Prep | Sections 8–9                           |
| Know what judges evaluate                     | Section 11                             |
| Prepare your demo and submission              | Sections 12–13                         |

## Important status note

The repository marks the nine rubric criteria as fixed, but the point weights are a draft proposal until organisers confirm them. The Round 2 / Round 3 score combination, dates and official submission form are also shown as TBA in the repository. Follow the latest official organiser announcement for those items.

## 1. What Round 3 is about

In Round 2, each participant built one specialist agent. In Round 3, your Pod brings its agents into a single workflow that follows a commerce unit through relevant checks and produces a final outcome supported by evidence.

### The core principle

Do not rebuild from scratch. Bring your existing Round 2 agent into the Pod repository, adapt it to the shared contract, connect it to the orchestrator and prove it works alongside the other stages.

Round 2 agents → Shared contracts → Pod orchestrator → Evidence trail → Final Commerce Outcome

## What a successful Pod demonstrates

- One complete, reproducible workflow that uses the Pod's real agents on its own units or captures.
- Clear routing: the correct stages run for the fulfilment route and return state.
- An orchestrator that owns workflow state; agents return judgments and evidence but do not set workflow status.
- Traceability from final decision to contributing records, individual checks and the exact input references.
- Honest handling of PASS, FAIL, UNCERTAIN, agent errors, invalid outputs and tenant isolation.

- A clear demo, accurate documentation, meaningful tests and a repository that a fresh clone can run.

## 2. First 30 minutes: get the starter running

The first task is to establish a shared, known-good baseline. Do this as a Pod before changing implementation code.

- Confirm whether the organisers assigned your Pod as Standard or Specialist. Do not choose the type yourselves.
- Choose one member to create the Pod's single shared fork or template copy of the official starter.
- Give the other four participants write access. Add the organiser account(s) requested for assessment with read access.
- Everyone clones the same Pod repository and uses the same main branch and pull-request workflow.
- Run setup, tests and a sample workflow on the shipped organiser stubs. Record any baseline issue before changing files.
- Fill `pod.json` and `.github/CODEOWNERS` together, then make a small, reviewed first commit.

```
git clone https://github.com/<owner>/<your-pod-repo>.git
cd <your-pod-repo>
make setup && make test && make run
make case UNIT=UNIT-0014 ORG=org_demo_alpha
```

Prerequisite: Python 3.11+ and Git. `make setup` creates a virtual environment, installs requirements and copies `.env.example` to `.env` if needed. Never commit `.env`.

### The stubs are scaffolding

The starter should run before integration using organiser stubs that replay synthetic Round 2 sample data. This proves the shared plumbing works. It does not prove your agent is integrated. Replace stubs one at a time and label every remaining stub honestly.

## 3. Know your Pod type

Every Pod has five participants. The organiser-assigned type determines which workflow and role assignment to use.

| Dimension          | Standard Pod                                   | Specialist Pod                                                                                           |
|--------------------|------------------------------------------------|----------------------------------------------------------------------------------------------------------|
| Participants       | Receiving, Prep, Pack, Returns, Recovery       | Receiving, Pack, Returns, Recovery + Specialist / Integration Engineer                                   |
| Agents in workflow | Five                                           | Four; Prep is absent                                                                                     |
| pod.json: pod_type | standard                                       | specialist                                                                                               |
| pod.json: flow     | orchestration/flow.json                        | orchestration/flow.specialist.json                                                                       |
| Specialist role    | Not applicable                                 | Coordinates orchestration and integration; supports adapters, hand-offs, tests, evidence and deployment. |
| Scoring treatment  | Members integrate and explain their own agent. | Not a sixth agent; contribution is assessed mainly at system level.                                      |

### Specialist Pod: configure, do not improvise

Set `pod\_type` to `specialist`, set `flow` to `orchestration/flow.specialist.json`, and replace the Prep member entry in `pod.json` with the Specialist / Integration Engineer role. The exact JSON shape is documented in the starter's `pod.json` notes and `FAQ.md`. Do not duplicate another agent to fill the missing Prep seat.

### What a Specialist does - and does not do

- Does: help the team align contracts and hand-offs, own the shared orchestration workstream as coordinator, drive end-to-end and failure tests, document decisions, and help deploy and demonstrate the integrated system.
- Does not: become a sixth agent-builder, work independently from the other four, or remove the requirement that Receiving, Pack, Returns and Recovery are integrated.

## 4. The commerce flow and routing

The starter represents the expected commerce journey. The logical route depends on the fulfilment channel and whether a return occurred; not every unit runs every stage.

| Stage                  | Purpose                                                                                                                                                  | When it runs                                                     |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|
| Receiving Manager      | Checks received supplier goods against the purchase order: identity, quantity, damage and quality.                                                       | First stage for each unit.                                       |
| Prep Manager           | Checks FBA prep requirements such as polybags, warnings, labels and handling marks. Evidence can support fee recovery.                                   | FBA units only in Standard Pods.                                 |
| Pack Manager           | Checks that the open box matches the order before sealing: correct items and quantities, nothing extra.                                                  | Merchant-fulfilled / 3PL (MFN) units.                            |
| Returns Manager        | Inspects a returned item, completeness and condition, then recommends disposition such as restock, refurbish, liquidate or dispose.                      | Only if the item was returned.                                   |
| Recovery Manager       | Reads accumulated evidence against channel fee reports and evaluates charge by charge whether evidence contradicts the charge, supports it or is silent. | Final decision stage.                                            |
| Final Commerce Outcome | A traceable final result derived from workflow state and accumulated evidence.                                                                           | Produced by the orchestrator, not copied directly from an agent. |

#### STANDARD POD

Receiving → FBA: Prep / MFN: Pack → If returned: Returns → Recovery → Final Outcome

#### SPECIALIST POD (no Prep)

Receiving → MFN: Pack → If returned: Returns → Recovery → Final Outcome

Specialist Pod FBA cases have no Prep evidence; inbound-defect charges are SILENT unless other valid evidence supports a decision.

Physically, hand-offs are coordinated and recorded by the orchestrator in the starter design. If your Pod changes the sequence, routing conditions or policy, document the reason and trade-offs in `docs/decisions.md` and update your own architecture documentation.

## 5. Who owns what

| Area                                               | Primary responsibility                     | Shared expectations                                                                                                                     |
|----------------------------------------------------|--------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| Receiving / Prep / Pack / Returns / Recovery agent | Named agent owner                          | Bring the Round 2 code, add provenance, implement the shared contract, preserve intended behavior, add own fixtures and contract tests. |
| Orchestration                                      | Coordinated by an agreed member; Pod-owned | Routes, validates, records evidence, updates state, retries, handles failures and uncertainty, derives final outcome.                   |
| Shared contracts and schemas                       | Pod-owned; changes discussed before merge  | Keep required envelope and schema compatibility. Extend agent-specific detail in `payload` or raise a contract issue.                   |
| Integration and end-to-end tests                   | Pod-owned                                  | Prove the agents work together on Pod fixtures and real units, not only organiser sample data.                                          |
| Docs, demo and submission                          | Pod-owned                                  | Keep build log, decisions, architecture, evaluation, run instructions and evidence current.                                             |

### The rule to remember

Agents decide about their stage and return evidence. The orchestrator owns workflow state and derives the final outcome. No agent may call another agent and silently mutate overall workflow state.

## 6. Bring your Round 2 agent into the Pod

- Copy your Round 2 implementation into `agents/<your-agent>`. Work in the Pod repo only; keep the submitted Round 2 repo intact.
- Add `PROVENANCE.md` identifying the original repository and source commit. Rewrite the stage README so it accurately describes the integrated version.
- Implement the input/output boundary expected by the starter. The normal in-process entry point is `handle(agent\_input) -> agent\_output`; HTTP mode must expose `GET /health` and `POST /run`.
- Update `agent.json` with the actual agent ID, owner, mode, URL if relevant, and honest implementation description. Do not label a stub as a real agent.

- Use the helpers in `shared/utls/records.py` and validate against schemas in `shared/schemas/`. Return a well-formed evidence record, not just a plain-text verdict.
- Ensure the agent refuses requests for a subject belonging to another `org\_id`; tenant isolation must also be enforced in storage.
- Run the agent contract tests on your own fixtures, then run `make test`. Add tests for the changes you made.

For Python agents, read the matching starter stub in `agents/<stage>/app.py`. For a separate service, see the README example for running an agent with Uvicorn and configuring its `mode` in `agent.json`. Other languages are possible if they honour the HTTP contract.

## 7. Build the orchestrator and integrate progressively

The starter already includes an orchestrator, flow configuration, storage interfaces and an API. Extend or replace parts where justified, but retain the required behavior and contracts.

### What the orchestrator must do

- Create and identify a workflow for the right organisation and unit.
- Choose the next applicable stage from the Pod's configured flow.
- Build Agent Input with the subject, stage-specific inputs, context, all prior evidence and applicable overrides.
- Invoke the agent and validate its output before accepting it (schema, stage, workflow ID, hashes and tenant).
- Record each evidence record and every workflow state transition; pass prior evidence forward to downstream stages.
- Handle routing, retries, timeouts, unavailable agents, invalid responses and UNCERTAIN according to explicit policies.
- Derive workflow status and Final Commerce Outcome from the evidence chain and overrides; do not simply copy an agent's recommendation.
- Keep enough diagnostics, audit information and evidence references to reproduce and explain a run.

### Recommended day-one integration method

- Baseline: `make setup && make test && make run` on the organiser stubs.
- Agree on contracts and routing before parallel implementation. Record important choices in `docs/decisions.md`.
- Replace one stub with one real agent and rerun contract, hand-off and end-to-end tests.
- Repeat stage by stage. Keep the workflow runnable throughout the build wherever possible.
- After all stages are integrated, test the clean route, claim route, return route, UNCERTAIN path, injected failure and wrong-tenant case.

## 8. Evidence, state and final decisions

The shared contract is what lets independently built agents become one system. The schema files in `shared/schemas/` are the machine-readable source of truth if a guide and schema disagree.

| Object      | Producer / owner | Purpose                                                                                        |
|-------------|------------------|------------------------------------------------------------------------------------------------|
| Agent Input | Orchestrator     | Workflow and subject identity, current stage, captures, context, prior evidence and overrides. |

| Object          | Producer / owner                          | Purpose                                                                                                                      |
|-----------------|-------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| Agent Output    | Agent                                     | Stage status, verdict, confidence, model/version, errors or recommendations, and the Evidence Record.                        |
| Evidence Record | Agent produces it; orchestrator stores it | A traceable per-stage, per-check account of inputs, verdicts, confidence, sources, hashes, upstream references and decision. |
| Workflow State  | Orchestrator only                         | Authoritative workflow status, stage results, transitions, evidence pointers and override history.                           |
| Final Outcome   | Orchestrator only                         | End-to-end decision derived from evidence and state, with reason, contributing records and human-review status.              |
| Error Record    | Agent or orchestrator                     | Makes a failure visible and diagnosable instead of hiding it.                                                                |

## Evidence record - the minimum mental model

Who? agent\_id + stage  
 For what? workflow\_id + subject.org\_id + subject\_id  
 When? captured\_at + produced\_at  
 What? checks[] (check\_key, verdict, confidence, expected/observed, detail)  
 Support? inputs[] + evidence\_refs[] + upstream\_refs[] + sha256  
 Decision? verdict + outcome + reason + needs\_human  
 How? model name/version, calls, cost and prompt version where applicable

Verdicts are `PASS`, `FAIL` and `UNCERTAIN`. A check that does not apply should be omitted rather than marked PASS. UNCERTAIN is not a low-confidence PASS: it means there is not enough reliable evidence for a conclusion.

### Do not overwrite history

Prior evidence must remain available. If a human override changes the effective decision, record the actor, reason and timestamp and reference the original evidence/decision. Downstream stages must be able to see the relevant prior evidence and overrides.

## 9. UNCERTAIN, errors and the missing Prep evidence

| Situation | Required behavior                                          | Never do this                       |
|-----------|------------------------------------------------------------|-------------------------------------|
| PASS      | Record evidence and continue to the next applicable stage. | Drop the evidence after reading it. |

| Situation                     | Required behavior                                                                                                                                                 | Never do this                                                 |
|-------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------|
| FAIL                          | Record the finding. The workflow policy determines routing and Recovery considers relevant evidence.                                                              | Treat every FAIL as a runtime exception.                      |
| UNCERTAIN                     | Preserve it as evidence. Route according to a documented policy and make human review available; record any override as a new decision referencing the original.  | Silently convert it to PASS or FAIL.                          |
| Timeout / unavailable / crash | Record a degraded/pending/error record and diagnostic. Retry transient failures as configured; if unresolved, the workflow must not be reported as success.       | Crash without a record or output CLEAN/COMPLETED.             |
| Invalid output / wrong tenant | Reject it and record the error. A cross-tenant result is a security event. Enforce tenant checks in both orchestrator and storage.                                | Accept invalid evidence or rely only on orchestrator checks.  |
| Stage not applicable          | Skip only because the flow's explicit routing condition says it does not apply, and preserve the reason where relevant.                                           | Skip because an agent is inconvenient or down.                |
| Specialist Pod missing Prep   | Do not fabricate Prep evidence. Recovery treats inbound-defect fee charges without Prep evidence as SILENT unless independent valid evidence supports a decision. | Claim a Prep-related fee using a guessed or invented verdict. |

In the starter, workflow statuses include `PENDING`, `IN\_PROGRESS`, `COMPLETED`, `FAILED`, `BLOCKED` and `RECOVERY\_REQUIRED`. A failure must not become success; a blocked UNCERTAIN case must keep its evidence and be visibly awaiting human review. Follow the orchestrator guide for the exact status transitions and policy behavior.

## 10. Security, cost and engineering rules

- Tenancy isolation first: include `org\_id` on every request, record and workflow. Test a second organisation and prove it cannot read or process another organisation's data.
- Batch model calls: make one model call per unit covering all relevant checks where practical and required; report `model.calls` and `model.cost\_usd` honestly.
- No secrets: never commit API keys, tokens, passwords, private keys or `.env`. Keep `.env.example` placeholders only. If a credential is exposed, revoke it immediately; deleting a commit is not enough.
- Use synthetic or properly authorised data. Do not put real customer, seller or supplier data in the repository without permission.
- Never log secrets, tokens or image bytes. Check browser tabs, terminal output and environment files before screen sharing.
- Keep `main` runnable. Use small feature branches and pull requests. Every PR is reviewed by another member; do not merge your own PR without review and never force-push to `main`.
- Do not weaken or delete tests merely to make CI green. Keep logs and error records structured enough to diagnose a failed run.
- Be precise in claims: a content hash is not automatically tamper-proof or tamper-evident; a starter stub is not your agent; a demo mock-up is not a running system.

# 11. How the Pod is evaluated

The Pod is evaluated as an integrated system. The nine criteria below are fixed in the starter rubric. The point allocation shown there is explicitly a draft proposal and remains TBA until the organisers confirm it. Round 2 performance remains part of the final result; the combination method is also TBA.

| # | Criterion                    | What judges want to see                                                                            |
|---|------------------------------|----------------------------------------------------------------------------------------------------|
| 1 | End-to-end integration       | Real units pass through the relevant stages to a traceable final outcome.                          |
| 2 | Agent interoperability       | Each agent honours the contract and downstream stages actually consume previous evidence.          |
| 3 | Orchestration                | The orchestrator owns state, routing, retries, transitions and final-outcome logic.                |
| 4 | Decision quality             | Sound, evidence-based judgments; sensible handling of uncertainty; no unsupported claims.          |
| 5 | Evidence & traceability      | A reviewer can trace the final outcome to records, checks and exact input references.              |
| 6 | Reliability & error handling | Failures, timeouts, invalid outputs and wrong-tenant cases are recorded and do not become success. |
| 7 | UX & demo                    | A human can understand the result, evidence and uncertainty; the demo shows the actual system.     |
| 8 | Engineering quality          | Clean-clone setup, tests, CI, security, reproducibility, honest limitations and cost reporting.    |
| 9 | Team collaboration           | Reviewed PRs, visible contributions, shared decisions and every member able to explain the system. |

## Draft values are not confirmed scores

The repository currently lists a proposed total of 100 points (15, 10, 10, 15, 15, 10, 10, 10, 5 in criterion order). Do not present these weights as final until an official organiser confirmation is issued.

## Standard vs Specialist scoring notes

- Standard Pods: individual agent integration is visible under relevant criteria as well as the shared system result.
- Specialist Pods: missing Prep is not penalised as an absent agent; their four-agent flow is assessed. The Specialist is not scored as having created a sixth agent and is read mainly through system-level contribution, especially criteria 1, 3, 5, 6, 8 and 9.

- Judges assess the submitted commit and the evidence that supports the claims. Running only organiser stubs, hand-picking one clean case or hiding uncertainty weakens the submission.

### Evidence that strengthens a score

- Your own units and captures, not only the starter samples.
- At least one clean case, one exception, one claim and one UNCERTAIN case preserved for human review and resolved with a recorded override.
- A deliberate failure injection that records the error and returns FAILED/INCOMPLETE rather than crashing or claiming success.
- Per-check TP/TN/FP/FN counts; false positives and false negatives shown separately; UNCERTAIN rate and method documented.
- Clear limitations, honest stub labels, green CI and reproducible steps from a clean clone.

## 12. Testing and demo

### Minimum testing plan

| Test level                 | Minimum proof                                                                                                                               |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Agent contract             | Every active agent returns schema-valid output for the Pod's own fixtures and refuses wrong-tenant requests.                                |
| Workflow state and routing | Each routing, status, override and policy change has tests.                                                                                 |
| End-to-end                 | At least one clean, one exception and one claim workflow using your own units.                                                              |
| UNCERTAIN                  | The original evidence remains intact, the workflow reaches BLOCKED / NEEDS_REVIEW where policy requires, and a human override is auditable. |
| Failure                    | Timeout, unavailable agent, invalid output and wrong tenant are recorded; failure is never presented as CLEAN/COMPLETED.                    |
| Clean clone / CI           | `make setup`, `make test` and the documented run command work from a fresh clone; CI is green on the final tag.                             |

### Recommended 10-minute demo

| Time      | Show                                                                                                            |
|-----------|-----------------------------------------------------------------------------------------------------------------|
| 0:00–1:00 | Problem and Pod type: one commerce chain, five members and roles.                                               |
| 1:00–2:30 | Architecture: agents, orchestrator, state ownership and key design choices.                                     |
| 2:30–5:00 | Run a complete workflow live from a case to the Final Commerce Outcome; label which agents are real or stubbed. |

| Time       | Show                                                                                                    |
|------------|---------------------------------------------------------------------------------------------------------|
| 5:00–6:00  | Trace evidence from final result to a check, upstream record and input hash.                            |
| 6:00–8:00  | Show an UNCERTAIN result and recorded override, plus an injected failure recorded as FAILED/INCOMPLETE. |
| 8:00–9:00  | Explain one claim supported by evidence and one SILENT charge left unclaimed.                           |
| 9:00–10:00 | Tests/CI, evaluation results, deployment and known limitations.                                         |

Every member should explain their own agent and decisions. The team should collectively explain the orchestrator. Prepare a recorded fallback, but label it honestly if used. Do not show mock-ups or hand-edited JSON as if they were live system output.

## 13. Final submission checklist

The exact deadline, submission window and official form are marked TBA in the current repository. Confirm them from the latest organiser announcement. Late submissions are not accepted under the carried-over rule unless organisers announce otherwise. Submit one final package per Pod.

- ☐ Pod fork is accessible to organisers; `pod.json` identifies the Pod, correct type, all members and orchestration coordinator.
- ☐ All required agents are integrated; any remaining stub is explicitly labelled in `agent.json` and the demo.
- ☐ `round3-final` tag points to the exact commit being submitted; everyone did a clean clone and retested that tag.
- ☐ `make setup && make test && make run` works, or the Pod README documents a tested equivalent; CI is green.
- ☐ README and `ARCHITECTURE.md` describe the Pod's actual system, not only the starter.
- ☐ `docs/decisions.md`, `docs/build-log.md` and `docs/evaluation.md` are up to date.
- ☐ Tests and evidence cover clean, exception, claim, UNCERTAIN/override, failure/resume and tenant-isolation paths.
- ☐ Demo recording/live plan is ready; deployment URL is reachable without personal account access, if applicable.
- ☐ No secrets in the repository or history; `.env.example` is complete and safe.
- ☐ All member names/handles and repository links are correct. Check links in a private window.
- ☐ LinkedIn post requirement and template are confirmed with organisers; the repository currently marks Round 3 confirmation/template as TBA.
- ☐ One member submits the repository/tag, demo, deployment URL (if any), evaluation doc and other fields required by the official form; save the confirmation.

## 14. Repository reading map

Use these source documents for implementation detail. Keep the linked repository open while you build; this handbook summarises but does not replace the schemas or source code.

| File / folder                                                         | What it answers                                                                                                     |
|-----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| <code>`START-HERE.md`</code>                                          | First read; objective, owner map, commands and required reading order.                                              |
| <code>`PARTICIPANT-GUIDE.md`</code>                                   | Individual and shared responsibilities; working sequence; agent integration expectations.                           |
| <code>`ARCHITECTURE.md`</code>                                        | System boundaries, state ownership, evidence flow, failure behavior and required Pod-specific architecture section. |
| <code>`INTEGRATION-GUIDE.md`</code>                                   | Agent hand-offs, request/response contracts and integration details.                                                |
| <code>`EVIDENCE-CONTRACT.md`</code> + <code>`shared/schemas/`</code>  | Exact fields, verdicts, evidence references, hashes, status and schema source of truth.                             |
| <code>`ORCHESTRATION-GUIDE.md`</code> + <code>`orchestration/`</code> | Routing, state transitions, error policies, persistence, overrides and resume.                                      |
| <code>`GITHUB-GUIDE.md`</code>                                        | One shared fork, branches, PR reviews, merge conflicts and repository security.                                     |
| <code>`RULES.md`</code> + <code>`FAQ.md`</code>                       | Non-negotiable rules and common questions.                                                                          |
| <code>`ROUND3-RUBRIC.md`</code>                                       | Nine fixed criteria, scoring notes, evidence of performance and judge reproduction.                                 |
| <code>`SUBMISSION-GUIDE.md`</code> + <code>`DEMO-GUIDE.md`</code>     | Deliverables, checklist, evaluation evidence and demo structure.                                                    |
| <code>`examples/`</code>                                              | Worked happy-path, UNCERTAIN, failure and end-to-end records.                                                       |
| <code>`tests/`</code>                                                 | Agent contracts, workflow state, end-to-end, examples and HTTP behavior.                                            |

## Quick reference: commands

| Command                   | Purpose                                                                                        |
|---------------------------|------------------------------------------------------------------------------------------------|
| <code>`make setup`</code> | Create the virtual environment, install dependencies and create <code>`.env`</code> if absent. |
| <code>`make test`</code>  | Run contract, workflow, end-to-end, failure, uncertainty, override, HTTP and example tests.    |
| <code>`make run`</code>   | Run all sample workflows; outputs are written under <code>`out/`</code> .                      |

| Command                                                         | Purpose                                                                               |
|-----------------------------------------------------------------|---------------------------------------------------------------------------------------|
| <code>`make case UNIT=UNIT-0014 ORG=org_demo_alpha`</code>      | Run and print a single workflow in full.                                              |
| <code>`make serve`</code>                                       | Start the orchestrator HTTP API on port 8100.                                         |
| <code>`pytest tests/integration/test_agent_contracts.py`</code> | Run the agent contract tests directly from the project virtual environment if needed. |

## The standard to aim for

### One Pod. One system. One traceable outcome.

A strong Round 3 build is not five agents shown side by side. It is a system whose workflow is coordinated, whose evidence can be followed, whose uncertainty is honest, whose failures are visible, and whose final outcome can be reproduced from the submitted code.

Official starter repository: <https://github.com/Cube-Build-A-Thon/cube-round3-pod>